

AccessMatrix AWS Key Management Service Implementation Guide - I-GA (AI)

Table of Contents

1. Preface - AccessMatrix AWS Key Management Service Implementation Guide.....	3
2. Introduction to AWS Key Management Service Implementation with AccessMatrix.....	5
3. Integration of AccessMatrix with AWS Key Management Service.....	9
4. AccessMatrix AWS KMS Key Provider Deployment.....	19
5. AccessMatrix Configuration.....	20

1. Preface - AccessMatrix AWS Key Management Service Implementation Guide

AWS Key Management Service (KMS) is a cloud-hosted managed service that allows you to create and manage cryptographic keys. You can control the encryption of data across AWS services. It gives you centralized control over the lifecycle and permissions of your cryptographic keys to protect your data. It provides a highly available key generation, storage, management, and more, to encrypt or digitally sign data within your applications. It uses FIPS 140-2 validated HSMs (Hardware Security Modules) to protect the confidentiality and integrity of your keys.

Intended Reader

The readers must possess some familiarity with AWS Key Management Service (KMS) concepts and operations.

Document Scope

This document shows you how to implement and integrate AccessMatrix with AWS KMS. AWS KMS supports both symmetric and asymmetric cryptographic keys and operations. AccessMatrix integrates with AWS KMS as a key provider to host its symmetric keys and perform symmetric cryptographic services inside AWS KMS.

This implementation guide will not explain how to manage resources on AWS KMS, as AWS cloud has its own vast and complete documentation. However, this document provides:

- Links to AWS documentation to provision/manage supporting resources (e.g. storage account, IAM user, AWS credentials and more) that must be prepared for this integration.
- AWS file commands for resources that are directly used by AccessMatrix (e.g. cryptographic keys).

For any feedback or questions regarding this document, contact doc@i-sprint.com.

Related Documents

For more information, see the following documents in the AccessMatrix Product Suite:

- [AccessMatrix Common Administration Guide](#)
 - [AccessMatrix Common Deployment Guide](#)
-

-
- [*UAS Pseudo E2EEA Implementation Guide*](#)

2. Introduction to AWS Key Management Service Implementation with AccessMatrix

The objective of this guide is to implement AccessMatrix integration with AWS Key Management Service (KMS). AccessMatrix uses AWS KMS as a key provider. You create an IAM user in Amazon Management Console using [AWS CLI \(Command-Line Interface\)](#), a unified tool to manage your AWS service. Refer to [Integration of AccessMatrix with AWS Key Management Service](#) for more information.

Glossary

This table lists some common terms that are used in this document. It is not intended as a comprehensive definition for each item.

Term	Description
Access Key Id	A unique identifier that's associated with a secret access key; the access key ID and secret access key are used together to sign programmatic AWS requests cryptographically.
ARN	Amazon Resource Name is a standardized way to refer to an AWS resource. For example: arn:aws:iam::123456789012:user/division_abc/subdivision_xyz/Bob.
KMS	AWS cloud-hosted Key Management Service is a managed service that simplifies the creation and control of encryption keys that are used to encrypt data.
IAM	AWS Identity and Access Management (IAM) is a web service that helps you securely control access to AWS resources. With IAM, you can securely control access to AWS services and resources for users in your AWS account. You use IAM to control who is authenticated (signed in) and authorized (has permissions) to use resources.
CLI	AWS CLI (Command Line Interface) is a unified downloadable and configurable tool for managing AWS services. Control multiple AWS services from the command line and automate them through scripts.
CMK	Customer Master Key is a customer-managed key, or an AWS managed key to be used inside AWS KMS to encrypt or decrypt of data or generated data keys.
HSM	Hardware Security Module, one of the options in KMS to host cryptographic keys and perform its operations.
Region	It represents the geographical data center locations. For example, ap-southeast1 (Singapore) and so on.
Resource	It refers to an entity that you can work within AWS. For example, an IAM user, CMK (Customer Master Key) and more.

Term	Description
Secret Access Key	A key that is used in conjunction with the access key ID to sign programmatic AWS requests cryptographically. Signing a request identifies the sender and prevents the request from being altered. You can generate secret access keys for your AWS account, individual IAM users, and temporary sessions.
Security Token Service	An AWS web service that enables you to request temporary credentials for users to access AWS resources. The temporary credentials are not stored with users and expire after a set amount of time, after which they cannot be used.

Implementation Checklist

Here is the list of items that must be prepared before integrating AccessMatrix with AWS KMS.

Item	Description
AWS account	You must register an AWS account which is a root account that allows full access to all resources in the account. This root user account is a member of an organization unit (OU). Note: To change an AWS support plan or to close an account, the root user has to sign in to the AWS Management Console with his or her email and password.
Billing Enabled	Billing will be enabled the moment you register an AWS account. Some features are free, while some are chargeable. You may refer to AWS Billing and Cost Management for more information. It allows up to 12 months with 20,000 free AWS KMS key requests and keys are chargeable at USD 1 per month of storage.
KMS Enabled	You have enabled AWS Key Management Service (KMS). Note: Amazon takes up to 24 hours to activate KMS.
IAM User	You have created an IAM user at the IAM Console by using your AWS account email address and password to sign in. Refer to this link to create IAM admin users.
AWS CLI (Command Line Interface) Enabled	You must download AWS CLI to make calls to your AWS services from the command line, and then carry out the configurations of CLI.
AccessMatrix	You have deployed AccessMatrix system on the Linux or Windows VM above in this list. Next, refer to AccessMatrix AWS KMS Key Provider Deployment for more details.

Implementation Planning

Security Consideration

Depending on the region you are at, keys created by AWS KMS are never transmitted outside of the AWS region in which they were created and can only be used in the region in which they were created. AWS KMS resources can be created in multiple locations. Refer to the global [Products and Services by Region](#) list to check if the AWS KMS managed service is available for your region.

Integration Consideration

These integrations use envelope encryption, where a data encryption key used by the AWS service to encrypt your data is protected under a customer master key (CMK) stored in AWS KMS. Customer managed CMKs give you full control over the lifecycle and permissions that determine who can use the key and under which conditions. As of version 5.6.6-GA, AccessMatrix need not be deployed in Linux VM on AWS Compute Engine (cloud) as it supports on-premise deployment of AccessMatrix that integrates with AWS KMS.

Authentication to AWS KMS

AWS KMS uses the [default credential provider chains](#) to look for AWS credentials, you must supply your created credentials in one of the following ways for AccessMatrix to communicate with AWS.

- Java System Properties: `aws.accessKeyId`, `aws.secretAccessKey`, `aws.region`
- Environment Variables: `AWS_ACCESS_KEY_ID`, `AWS_SECRET_ACCESS_KEY`, `AWS_REGION`
- Default credentials profiles file: `./aws/credentials` and `./aws/config`

You also use the `BasicAWSCredentials` method bundled inside the AWS SDK to supplement the credentials. For example:

Java

```
BasicAWSCredentialsaws=newBasicAWSCredentials(accessKeyId,  
secretAccessKey);
```

Out of Integration Scope

Asymmetric encryption/decryption keys and operations are out of scope in this integration of AccessMatrix system to AWS KMS.

Key Management Functions

You can perform the following key management functions in the Amazon Management Console:

- Create symmetric keys for local use within your applications
- Define which IAM users and roles can manage keys
- Define which IAM users and roles can use keys to encrypt and decrypt data
- Choose to have keys that were generated by the service to be automatically rotated on an annual basis
- Temporarily disable keys so no one can use them
- Re-enable disabled keys
- Schedule the deletion of keys that you no longer use



Note: You need to wait for 7-30 days for the termination of keys to take effect.

Performance Consideration

Customer master keys are the primary resources in AWS KMS used to encrypt and decrypt data. AccessMatrix uses the AWS KMS owned CMKs (Customer Master Keys) in AWS KMS to encrypt and decrypt your data. Unlike the Customer Master key or Customer Managed Key, AWS owned CMKs never leave the AWS KMS FIPS validated hardware security modules unencrypted, you do not need to decrypt the data outside AWS KMS.

Refer to [AWS documentation](#) for more information about the types of CMK as needed.

3. Integration of AccessMatrix with AWS Key Management Service

AccessMatrix uses AWS Key Management Service (KMS) as a key provider, which means it uses AWS KMS to store symmetric keys and perform cryptographic operations (encryption/decryption) inside it.

AccessMatrix does not provide key management functions such as creating the keys, destroying the keys, rotating the keys and others to AWS KMS. All of those key management/operations are to be done directly in the Amazon Management Console (either using AWS shell or AWS CLI).

The integration steps will also differ slightly based on which method is used to allow AccessMatrix to access AWS KMS. You can decide between using role-based access or access key-based access:

- **Role-based Access**

This method uses the AWS Security Token Service (STS) to create [temporary security credentials \(IAM roles\)](#) that allow access to AWS resources. These credentials are generated dynamically and provided to the user when requested; they are not stored with the user. Once they have expired, AWS will no longer recognize them or allow access based on them. This is the recommended access method for greater security.

- **Access Key-based Access**

This method uses [access keys](#), which are long-term credentials that allow access to AWS resources. They do not expire and must be manually revoked. It is strongly recommended that you do not to use this method; instead, use the more secure role-based access.

i **Note:** Even if role-based access is used, access keys are still required for the API call to create the initial credentials provider. Therefore, you should not skip any of the access key-related steps below unless specified.

Create an IAM Admin User

When you first create an AWS account, you begin with a single sign-in identity that has complete access to all AWS services and resources in the account. This identity is called the AWS account root user and is accessed by signing in with the email address and password that you used to create the account. Learn about the [IAM Console and Sign-In page](#) first, and then follow the steps [here](#) to create your IAM admin user.

i **Note:** It is recommended that you do not use the root user for your everyday tasks, even the administrative ones. Instead, adhere to the best

practice of using the root user only to create your first IAM user. Then securely lock away the root user credentials and use them to perform only a few account and service management tasks. Refer to [AWS IAM user guide](#) for more information as needed.

Manage Access Keys

Access Keys are long-term credentials for IAM users. They are used to sign requests to AWS CLI or API. You can create access keys via the following interfaces. Select each of them below to learn how to create, modify and delete the access keys in the AWS documentation.

- [Amazon Management Console](#)
- [AWS CLI](#)
- [AWS API](#)



Note: An access key consists of an access key Id and a secret access key. You download the access keys in a **Download.csv** file. Store the file in a secure location. Do not provide any third party with the access keys.

Role-based management of access keys

AWS IAM Roles Anywhere is a service provided by AWS that allows you to obtain temporary security credentials within IAM for workloads running outside of AWS. With AWS IAM Roles Anywhere, you can use IAM policies and roles created within AWS IAM to manage access keys stored outside of AWS for greater security.

To manage your access keys in this manner, you first need to generate the access keys and store them in a secure environment (e.g. as environment variables, or in a .Credentials file) outside of AWS. Subsequently, you can set up IAM Roles Anywhere to manage the access keys.

The setup and configuration of IAM Roles Anywhere for this purpose is beyond the scope of this guide. However, you can refer to the following AWS documents to learn more about it:

- [AWS IAM Roles Anywhere](#)
- [Creating temporary security credentials in IAM](#)

Configure AWS Credentials using AWS CLI

Prerequisite:

-
- You have already installed and configured AWS CLI (Command-Line Interface).

Use the command `aws configure` to call AWS service and API after you have created the IAM Admin user, granted policies and created the access keys. For example:

```
aws configure
AWS Access Key ID [*****B66I]:
AWS Secret Access Key [*****HOFF]:
Default region name [ap-southeast1]: ap-southeast-1
Default output format [json]: json
```

Alternatively, you can add these credentials in the config file in the folder `C:\Users\Username\.aws` and save it.

AWS KMS will lookup for the user credentials in the following sequences.

- **Java system properties** – `aws.accessKeyId` and `aws.secretAccessKey`. It uses the `SystemPropertyCredentialsProvider` class to load these credentials.
- **Environment variables** – `AWS_ACCESS_KEY_ID` and `AWS_SECRET_ACCESS_KEY`. It uses the `EnvironmentVariableCredentialsProvider` class to load these credentials.
- **The default credential profiles file** – `./aws/credentials`.

Create a Customer Master Key (CMK)

The Customer Master Key (CMK) is a logical representation of a master key that includes metadata like the keyID, creation date, description and key date, as well as key materials for encryption/decryption purposes. AWS KMS uses symmetric CMKs to protect your data. See how to create symmetric CMKs [here](#). Notice you can create the symmetric keys by using the Amazon Management Console or the AWS KMS API.

In AWS CLI, use the following command `aws kms create-key` to create the symmetric CMKs, and the command `aws kms list-keys` to display all the CMKs.



Notes:

- CMKs are stored in Amazon HSM and will never leave it unencrypted.
- Storing of CMKs in Amazon HSM is charged at USD 1 / key each month, whether it is in-used or unused. The scheduled keys for deletion are uncharged.

Key Policies to Control Access to CMKs

In AWS KMS, [key policies](#) are used to control access to CMKs. Some permissions are needed to carry out encryption and decryption operations. You can grant the permissions by using the AWS Console or AWS CLI.

Important Note: When using AWS CLI to add the key policies, the IAM user needs to be granted the following policies:

- kms:PutKeyPolicy
- kms:GetKeyPolicy

Typically, if the permissions are granted via CLI, the command takes in a policy JSON file that contains the required privileges. Access can be controlled because the key policies are set on the CMK. For example, use **key_policy.json** to encrypt or decrypt in AccessMatrix on a CMK to another user.

The key administrator's statement allows the following permissions:

- kms:DescribeKey
- kms:ListKeys (Optional)

i Notes:

- The ListKeys permission is optional; to ensure that minimal privileges are granted for greater security, you may choose not to allow it.
- Depending on whether this permission is granted, there may be minor differences in the AccessMatrix configuration process. Refer to [AccessMatrix Configuration - Configure Wrapper Key](#) for more information.

- kms:Encrypt
- kms:Decrypt
- kms:ReEncrypt
- kms:GenerateDataKey*

For the role-based access method, an additional permission, `sts:AssumeRole`, is also required. This permission must be granted to an IAM role, which should then be assigned to the CMK. The full instructions for this are detailed in the [next section](#).

The following is a sample **key_policy.json** file used to enable access key-based access:

```
JSON
```

```
{
  "Version" : "2012-10-17",
  "Id" : "key-default-1",
  "Statement" : [
    {
      "Sid" : "Enable IAM User Permissions",
      "Effect" : "Allow",
      "Principal" : {
        "AWS" : "[ARN of root user]"
      },
      "Action" : "kms:*",
      "Resource" : "*"
    },
    {
      "Sid" : "Allow Use of Key and
Encryption/Decryption",
      "Effect" : "Allow",
      "Principal" : {
        "AWS" : "[ARN of user this policy is meant
for] "
      },
      "Action" : [
        "kms:DescribeKey",
        "kms:ListKeys",
        "kms:Encrypt",
        "kms:ReEncrypt*",
        "kms:GenerateDataKey*",
        "kms:Decrypt"
      ],
      "Resource" : "*"
    }
  ]
}
```



Note: See [Managing Access to AWS KMS CMKs](#) for more information.

To add the key policy to a CMK (replace the placeholder names as appropriate):

```
aws kms put-key-policy --policy-name default --key-id [CMK KeyID] --policy
file://key_policy.json
```

Configure IAM Role (Role-based Access Only)

For the role-based access method, an additional permission, `sts:AssumeRole`, must be granted to an IAM role. The IAM role must then be assigned to the CMK. This section describes how to perform these configurations to enable role-based access to AWS:

- [AWS CLI Configuration](#)
- [Management Console Configuration](#)



Note: You can refer to the following links for more information:

- [Creating an IAM policy](#)
- [Creating an IAM role](#)
- [Assigning an IAM policy](#)
- [Add key user for KMS key](#)

AWS CLI Configuration

To configure IAM Role using AWS CLI, perform the following steps:

1. Set the following environment variables for easier reference:

```
RoleName=KMS-Role
```

```
PolicyName=KMS-Policy
```

2. Create the policy document for the role. This document should contain the ARN of a user (as a principal for the role) to assume this role, as well as the permission `sts:AssumeRole`. For example:

policydocument.json
<pre>JSON { "Version": "2012-10-17", "Statement": [{ "Effect": "Allow", "Principal": { "AWS": "arn:aws:iam::12345678910:root" }, "Action": "sts:AssumeRole", "Condition": {} }] }</pre>

3. Create a role with the above document:

```
$ aws iam create-role --role-name $RoleName --assume-role-policy-document
file://policydocument.json
```

The following is an example of the output should be observed:

Code

```
{
  "Role": {
    "Path": "/",
```

```

        "RoleName": "kms-role",
        "RoleId": "AROAJ4EHWNV6CFK5L47S",
        "Arn": "arn:aws:iam::12345678910:role/kms-
role",
        "CreateDate": "2023-08-22T02:57:38+00:00",
        "AssumeRolePolicyDocument": {
            "Version": "2012-10-17",
            "Statement": [
                {
                    "Effect": "Allow",
                    "Principal": {
                        "AWS":
"arn:aws:iam::12345678910:root"
                    },
                    "Action": "sts:AssumeRole",
                    "Condition": {}
                }
            ]
        }
    }
}

```

4. Create a policy document that contains the permissions for KMS. For example:

kmspolicy.json	
JSON	
	<pre> { "Version": "2012-10-17", "Statement": [{ "Sid": "KMS", "Effect": "Allow", "Action": ["kms:EnableKeyRotation", "kms:EnableKey", "kms:GetPublicKey", "kms:Decrypt", "kms:GetKeyPolicy", "kms:GenerateDataKeyWithoutPlaintext", "kms:GenerateDataKeyPairWithoutPlaintext", "kms:DisableKey", "kms:GenerateDataKeyPair", "kms:ReEncryptFrom", "kms:ListKeys", "kms:Encrypt", "kms:GenerateDataKey", "kms:CreateAlias", "kms:ReEncryptTo", "kms:DescribeKey"], "Resource": "*" }], } </pre>

kmspolicy.json

```
{
  "Sid": "STS",
  "Effect": "Allow",
  "Action": "sts:AssumeRole",
  "Resource": "arn:aws:iam::12345678910:role/root"
}
```



Note: This document takes the place of the policy document **key_policy.json** mentioned in the [Key Policies to Control Access to CMKs](#) section.

5. Create a policy with the above document:

```
$ aws iam create-policy --policy-name $PolicyName --policy-document
file://kmspolicy.json
```

The following is an example of the output that should be observed:

Code

```
{
  "Policy": {
    "PolicyName": "kms-policy",
    "PolicyId": "ANPAQJ4EHWNVVI46C3JUM",
    "Arn": "arn:aws:iam::12345678910:policy/kms-
policy",
    "Path": "/",
    "DefaultVersionId": "v1",
    "AttachmentCount": 0,
    "PermissionsBoundaryUsageCount": 0,
    "IsAttachable": true,
    "CreateDate": "2023-08-22T02:58:09+00:00",
    "UpdateDate": "2023-08-22T02:58:09+00:00"
  }
}
```

6. Attach this role policy to the role created earlier:

```
aws iam put-role-policy --role-name $RoleName --policy-name $PolicyName --policy-
document file://kmspolicy.json
```

7. Retrieve the ARN of the role for AccessMatrix's use:

```
$ aws iam get-role --role-name $RoleName --query Role.Arn --output text
```

The following is an example of the output that should be observed:

Code

```
arn:aws:iam::12345678910:role/kms-role
```

Management Console Configuration

To configure IAM Role using the Management Console, perform the following steps:

i **Note:** The following steps assume that you have already created a **key_policy.json** file and added it to the CMK, as described in the [Key Policies to Control Access to CMKs](#) section. This section only details the creation and configuration of the IAM role with the `sts:AssumeRole` permission.

1. Create a new policy:
 - a. Access the AWS Management Console and navigate to the IAM console.
 - b. On the left pane, select **Policies**, then choose **Create Policy**. The **Specify permissions** page is displayed.
 - c. Under the **Service** section, search for the "STS" service and add it.
 - d. Under **Access Level** in the **Actions Allowed** section, expand the **Write** setting and add the **AssumeRole** permission.
 - e. Under the **Resources** section, click **Add ARN**. The **Specify ARNs** dialog is displayed.
 - f. Click the **This account** radio button, then tick the **Any role name with path** and **Any resource** checkboxes.
 - g. Click **Add ARNs**.
 - h. Click **Next**. The **Review and create** page is displayed.
 - i. Enter an appropriate **Policy Name** (e.g. "kms-policy") and click **Create policy** to complete the policy creation process.
 2. Create the IAM role and assign the policy:
 - a. On the left pane, select **Roles**, then choose **Create role**. The **Select trusted entity** page is displayed.
 - b. Under **Trusted entity type**, choose the **AWS Service** role type.
 - c. Under **Use case**, search for and add the "STS" service.
 - d. Click **Next**. The **Add permissions** page is displayed.
 - e. Select the checkbox next to the previously created policy (e.g. "sts-policy"), then click **Next**. The **Name, review, and create** page is displayed.
 - f. Enter an appropriate **Role name** (e.g. "kms-role").
-

-
- g. Click **Create role** to complete the process.
 - h. On the page of the newly created role, in the **Summary** section, note down the value under **ARN** (e.g. "arn:aws:iam::123456789101:role/kms-role").
3. Add the role as key user in KMS:
- a. Navigate to the KMS console.
 - b. On the left pane, select **Customer managed keys** and navigate to the CMK created in the [earlier section](#).
 - c. In the **Key policy** tab, under the **Key users** section, click the **Add** button.
 - d. Select checkbox for the previously created IAM role (e.g. "AMRole") and click **Add**.
-

4. AccessMatrix AWS KMS Key Provider Deployment

This part assumes AccessMatrix has been deployed properly on AWS KMS. You can confirm it by opening AdminConsole, visit <http://<hostname:port>/am5> (or <https://<hostname:port>/am5>).

AWS KMS Key Provider Plugin Deployment

The AWS Key Management Service (KMS) key provider is implemented as plugin. Thus, you need to deploy the plug-in into AccessMatrix folder by following these steps:

1. Copy **am-awskmskprov-core.jar** file from `<ACMATRIX_ROOT>\sdk\lawskmskprov\` folder to `<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\lib\` folder.
2. Copy `<ACMATRIX_ROOT>\sdk\lawskmskprov\lib\*.jar` files to the `<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\lib\` folder.
3. Copy **am-awskmskprov-plugin.jar** file from `<ACMATRIX_ROOT>\sdk\lawskmskprov\` folder to `<ACMATRIX_ROOT>\tomcat\plugins\` folder.
4. Copy **field-definitions-AWSKMSKeyProvider.xml** and **wpecustom-AWSKMSKeyProvider.properties** from `<ACMATRIX_ROOT>\sdk\lawskmskprov\res\` folder to `<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes\` folder.
5. Rename **wpecustom-AWSKMSKeyProvider.properties** in the folder `<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes\` to **wpecustom.properties**.
6. Add the following line to the file **amsystem.properties** in the folder `<ACMATRIX_ROOT>\tomcat\webapps\am5\WEB-INF\classes\`.

Properties (.properties files)

```
am.field_definition.file.0=field-definitions-  
AWSKMSKeyProvider.xml
```

7. Start AccessMatrix server, check **am.log** file (`<hostname>_am.log`) to see if AWS KMS Key Provider plugin is loaded. Find this keyphrase Plugin loaded: AWSKMSKeyProviderPlugin, e.g.

```
2019-12-23 15:47:47.885 [main] INFO - (PluginManager.java:331) Loading plugin instance,  
spld=AWSKMSKeyProviderPlugin, version=0.1,  
instance=com.isprint.am.core.impl.ext.plugin.AWSKMSKeyProviderPlugin@6193f60c, loadOrder=10  
2019-12-23 15:47:47.885 [main] INFO - (AWSKMSKeyProviderPlugin.java:29) Plugin loaded:  
AWSKMSKeyProviderPlugin - 0.1
```

5. AccessMatrix Configuration

This part assumes the AccessMatrix AWS KMS Key Provider plugin has been deployed/installed successfully. You can confirm it by navigating to **Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider**, click the **Create** button and see if "AWS KMS Key Provider" is listed in the **Implementation** drop-down list.

To configure the key provider and encryption keys in AccessMatrix AWS KMS, you should have completed the steps detailed in the [Integration of AccessMatrix with AWS Key Management Service](#) and [AccessMatrix AWS KMS Key Provider Deployment](#) and have generated an access key Id, secret access key, and a region.

Create Key Provider and Encryption Key

Create a key provider of AWS KMS Provider implementation. Refer to [AccessMatrix Common Administration Guide - Configure Key Provider](#) for more information.

Create encryption keys that will be used for various purposes based on the AWS KMS Key Provider created. AccessMatrix AWS KMS supports symmetric keys for the following purposes:

- Attribute encryption
- Key derivation
- Key wrapping
- Pin encryption
- Token encryption



Notes:

- Refer to [AccessMatrix Common Administration Guide - Configure Encryption Key](#), for an explanation on each key's purpose.
- If the use case is for PseudoE2EEA, refer to [UAS Pseudo E2EEA Implementation Guide](#).

Setup Steps

This section shows you how to configure the AccessMatrix server to integrate with AWS KMS on the Admin Console. Before you start, it is assumed that you have completed the pre-requisites.

- **Configure AWS KMS Key Provider**

This key provider uses the client libraries as the crypto library in AWS KMS. Use the "AWS KMS Key Provider" implementation in AccessMatrix server to integrate with AWS KMS. Refer to [Configure AWS KMS Key Provider](#) for more details.

- **Configure Wrapper Key**

This symmetric wrapping key supports the encryption operations by using the public portion of the key to protect and wrap the symmetric keys in AWS KMS. This operation requires the wrap key permission. Refer to [Configure Wrapper Key](#) to learn how to create the wrapper key in the AccessMatrix Admin Console.

- **Configure PIN Encryption Key (PEK)**

PEK (PIN Encryption Key) is the key that uses AES (Advanced Encryption Standard) to encrypt the PIN in the AccessMatrix security registry. The wrapping key wraps PEK. PEK is randomly generated based on the algorithm provided by the JCE (Java Cryptographer Extension). Refer to [Configure PIN Encryption Key \(PEK\)](#) to learn how to use the wrapper key to protect this Pin Encryption Key.

Configure AWS KMS Key Provider

Upon successfully deploying the AccessMatrix AWS KMS Key Provider plugin, the "AWS KMS Key Provider" will be created by default in the Admin Console. Depending on whether you are using [role-based access](#) or [access key-based access](#), the configuration steps here will differ slightly:

Role-based Access	1. Navigate to Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider on the left pane. 2. Click Create and select "AWS KMS Key Provider" in the Implementation drop-down list. 3. Click Next. 4. Enter a Key Provider Id (e.g. "AWSKMSKeyProv") and a Key Provider Name (e.g. "AWSKMSKeyProv") and enter its description. 5. Specify the AWS region (e.g. "ap-southeast-1"). 6. Specify the IAM Role ARN (e.g. "arn:aws:iam::123456789101:role/kms-role"). This is the ARN of the role created in this section, which you previously noted down. 7. Specify the IAM Role Duration (e.g. "900"). This is the duration that the temporary credentials will last before expiry (in seconds). It can be any value from 900 to 3600. 8. Click Save. A new key provider is created.
	 Note: You must ensure that the attributes under Access Key-based Access (i.e. Access Key ID and Secret Access Key) are empty. If these values are configured, they will take precedence over the role-based access configurations.

Access Key-based Access	1. Navigate to Administration Dashboard > Security Policy > Key Management > Key Manager > Key Provider on the left pane. 2. Click Create and select "AWS KMS Key Provider" in the Implementation drop-down list. 3. Click Next. 4. Enter a Key Provider Id (e.g. "AWSKMSKeyProv") and a Key Provider Name (e.g. "AWSKMSKeyProv") and enter its description. 5. Specify the AWS region (e.g. "ap-southeast-1"). 6. Specify the Access Key Id (e.g. "AKIA54KH25v0YaGrB66G"). 7. Specify a value for one of the following attributes only: a. Secret Access Key - Use this attribute if you are not using this AWS KMS key provider as the default system key provider's Attribute Encryption Key (AEK). Specify the secret access key (e.g. "yyVBULaFF3j0aA+fp3ep3mQMxMFVdscFQkgRzHGTT"). b. Secret Access Key (System Encrypted) - Use this attribute if you are using this AWS KMS key provider as the default system key provider's AEK. Specify the secret access key (e.g. "yyVBULaFF3j0aA+fp3ep3mQMxMFVdscFQkgRzHGTT").  Important Note: It is important to choose the correct attribute to prevent critical issues from arising. This secret access key is protected by the AEK in AccessMatrix. If this AWS KMS key provider's key is set as the default system AEK, it will be used to encrypt itself, leading to a circular dependency and critical issues. If you intend to use the AWS KMS key as the default system AEK, leaving the Secret Access Key attribute blank and instead configuring Secret Access Key (System Encrypted) causes this secret access key to be encrypted by the default system key (AEK1) instead of this AWS KMS key to avoid such issues.  Notes: ▪ If a value is specified for both attributes, Secret Access Key (System Encrypted) will take precedence. ▪ For more information on the attribute encryption key, refer to AccessMatrix Common Administration Guide - Key Management - Types of Encryption Keys. 8. Click Save. A new key provider is created.
-------------------------	--

Configure Wrapper Key

1. Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key** on the left pane.
2. Click **Create** and select the newly created key provider (e.g. "AWSKMSKeyProv") from the **Key Provider** drop-down list.

Notice there is only one available **Encryption Key Template**, "Symmetric key (secret key)", as

AccessMatrix system integrates with AWS KMS to perform mainly symmetric operations for key encryption.

3. Click **Next**.
4. Enter an **Encryption Key Id** (e.g. "AWrapKey") and an **Encryption Key Name** (e.g. "AWrapKey") and enter its description.
5. Under the *** Customer Master Key (CMK ID)** field, specify the value of the master key to be used:
 - a. If AccessMatrix is able to detect and fetch any CMKs from AWS , you will see a drop-down list with all the available CMK IDs. Select the appropriate master key.
 - b. If AccessMatrix is unable to detect and fetch any CMKs from AWS, you will see a text input field. You can manually input the CMK ID into the field.



Note: There are various reasons AccessMatrix might be unable to fetch CMKs from AWS, such as:

- A lack of permissions such as the ListKeys permission.
- The region(s) specified during CMK creation is/are not connected to the AccessMatrix system.
- Connection failure.

You can consult the AM log for more information.

6. Next, select "Key Wrapping" from the ***Purpose** drop-down list under the **Attributes** tab.
7. Click **Save**. A Wrapper Key is created.



Note: You can click the **Test KeyInfo** and **View Public Key** buttons to check if this key is created successfully.

Configure PIN Encryption Key (PEK)

1. Navigate to **Administration Dashboard > Security Policy > Key Management > Encryption Key** on the left pane.
2. Click **Create** and select the newly created key provider (e.g. "AWSKMSKeyProv") from the **Key Provider** drop-down list.

Notice there is only one available **Encryption Key Template**, "Symmetric key (secret key)", as AccessMatrix system integrates with AWS KMS to perform mainly symmetric operations for key encryption.

3. Click **Next**.
-

-
4. Enter an **Encryption Key Id** (e.g. "PEK2") and an **Encryption Key Name** (e.g. "PEK2") and enter its description.
 5. Under the *** Customer Master Key (CMK ID)** field, specify the value of the master key to be used:
 - a. If AccessMatrix is able to detect and fetch any CMKs from AWS , you will see a drop-down list with all the available CMK IDs. Select the appropriate master key.
 - b. If AccessMatrix is unable to detect and fetch any CMKs from AWS, you will see a text input field. You can manually input the CMK ID into the field.



Note: There are various reasons AccessMatrix might be unable to fetch CMKs from AWS, such as:

- A lack of permissions such as the ListKeys permission.
- The region(s) specified during CMK creation is/are not connected to the AccessMatrix system.
- Connection failure.

You can consult the AM log for more information.

6. Select "Pin Encryption" from the ***Purpose** drop-down list under the **Attributes** tab.
7. Click **Save**. A Pin Encryption Key is created.



Note: You can select **Test KeyInfo** and **View Public Key** buttons to check if this key is created successfully.
